

# Reading and writing networks

Simon Frost

## Table of contents

|    |                                  |   |
|----|----------------------------------|---|
| 1  | Overview                         | 1 |
| 2  | Setup                            | 2 |
| 3  | The intermediate representation  | 2 |
| 4  | Comparing the four files         | 3 |
| 5  | A brute-force sanity check       | 3 |
| 6  | Writing                          | 4 |
| 7  | Format detection and strict mode | 5 |
| 8  | The package's own JSON           | 5 |
| 9  | Summary                          | 6 |
| 10 | References                       | 6 |

## 1 Overview

`BayesianNetworkFormats.jl` reads six interchange formats (Netica `.dne`, GeNIe `.xdsl`, HUGIN `.net`, BIF, DSC and UAI) into a single intermediate representation, `NetworkIR`, and writes that representation back out in any of them. It depends on nothing else in the `ecorecipes` ecosystem, so it can be used on its own.

This vignette reads the classic *asia* network ([Lauritzen and Spiegelhalter 1988](#)), taken from the `bnlearn` repository ([Scutari 2010](#)), in four formats, checks that the four files describe the same model, and writes it out in the remaining formats.

## 2 Setup

Only `BayesianNetworkFormats.jl` is needed; it has no ecosystem dependencies. The four *asia* files are fixtures shipped with the package.

```
using BayesianNetworkFormats

paths = Dict{ext => relpath(fixture_path("${ext}/asia.${ext}"))
           for ext in ("bif", "net", "dsc", "uai")}
asia = Dict{ext => read_network(path) for (ext, path) in paths}
asia["bif"]
```

```
NetworkIR("unknown": 8 chance; format=:bif)
```

`fixture_path` returns the absolute path of a file committed under `test/fixtures/`, so the same models are available to downstream packages and to these vignettes. The reader records the path it is given as the network's source, so the vignette passes it through `relpath`, relative to the working directory, and the source shown below is the same on every machine.

## 3 The intermediate representation

A `NetworkIR` holds a vector of `IRVariables` in file order. Each variable has an id, a kind (chance, decision or utility), ordered states, ordered parents, and a table.

```
dysp = variable(asia["bif"], :dysp)
```

```
IRVariable(:dysp, chance, states=["yes", "no"], parents=[:bronc, :either])
```

```
dysp.parents, dysp.states
```

```
([:bronc, :either], ["yes", "no"])
```

The table of a chance node has one axis per parent, in parent order, followed by the axis of the node itself, and every row sums to one. Here `dysp` has parents `bronc` and `either`, so `dysp.table[2, 1, :]` is the distribution of `dysp` given `bronc = no` and `either = yes`:

```
dysp.table[2, 1, :]
```

```
2-element Vector{Float64}:
 0.7
 0.3
```

```
size(dysp.table)
```

```
(2, 2, 2)
```

This convention is fixed by ADR 0002 of the ecosystem; the vignette *Axis conventions* goes through how each file format lays its numbers out and how they are converted.

## 4 Comparing the four files

The .bif, .net and .dsc files are verbatim from bnlearn ([Scutari 2010](#)); the .uai file was produced by this package's writer. `differences` lists everything that differs between two IRs and `isequivalent` is its boolean form. The network name is not part of every format (the HUGIN file has none), so it is excluded from the comparison:

```
for ext in ("net", "dsc", "uai")
    println(rpad(ext, 5), isempty(differences(asia["bif"], asia[ext]; name=false)) ?
        "same model" :
            differences(asia["bif"], asia[ext]; name=false))
end
```

```
net  same model
dsc  same model
uai  same model
```

The UAI format only stores cardinalities and numbers. Variable and state names come from a sidecar file, `asia.uai.names`, that the writer produces and the reader picks up when present:

```
println(read(paths["uai"] * ".names", String))
```

```
# variable and state names for the UAI file; written by BayesianNetworkFormats.jl
network asia
asia yes no
tub yes no
smoke yes no
lung yes no
bronc yes no
either yes no
xray yes no
dysp yes no
```

## 5 A brute-force sanity check

`joint_distribution` multiplies every table over every joint configuration. It is slow (exponential in the number of variables) but has no room for an axis mistake, which makes it the oracle for the whole package:  $P(\text{dysp} = \text{yes}) = 0.4360$  is the published value for *asia* ([Lauritzen and Spiegelhalter 1988](#)).

```
marginal(asia["bif"], :dysp)
```

```
2-element Vector{Float64}:
 0.43597059999999993
 0.5640293999999999
```

## 6 Writing

`write_network` picks the format from the extension (or takes a format singleton) and validates the IR first. Reading the result back gives an equivalent IR:

```
dir = mktempdir()
for fmt in (NeticaDNE(), GeNIeXDSL(), HuginNET())
    out = joinpath(dir, "asia." * string(format_name(fmt)))
    write_network(out, asia["bif"])
    back = read_network(out)
    println(rpad(basename(out), 10), isequivalent(back, asia["bif"]) ? "round trip
    identical" :
        differences(back, asia["bif"]))
end
```

```
asia.dne round trip identical
asia.xdsl round trip identical
asia.net round trip identical
```

The Netica writer lays the table out the way Netica does ([Norsys Software Corp. 2024](#)), with one parenthesis level per parent and a comment naming the parent configuration on every row:

```
lines = readlines(joinpath(dir, "asia.dne"))
i = findfirst(=="node dysp {"), lines)
println(join(lines[i:(i + 12)], "\n"))
```

```
node dysp {
  kind = NATURE;
  discrete = TRUE;
  chance = CHANCE;
  states = (yes, no);
  parents = (bronc, either);
  probs =
    // yes no // bronc either
    (((0.9, 0.1), // yes yes
      (0.8, 0.2)), // yes no
      ((0.7, 0.3), // no yes
      (0.1, 0.9))); // no no
};
```

## 7 Format detection and strict mode

`detect_format` looks at the extension and then at the first bytes of the file:

```
detect_format(paths["net"]), detect_format(paths["bif"])
```

```
(HuginNET(), BIF())
```

Some node types have no counterpart in the IR: GeNIe equation and noisy-MAX nodes, Netica continuous nodes without discretisation levels, HUGIN continuous nodes. In strict mode (the default) they raise an `UnsupportedNodeError` naming the node:

```
try
  read_network(fixture_path("xdsl/equation_node.xdsl"))
catch e
  showerror(stdout, e)
end
```

```
UnsupportedNodeError: node Runoff of type equation is not supported by the xsdl
format
```

With `strict=false` the node, and every node that depends on it, is dropped and recorded in `extras[:skipped]`, so the remaining model is still consistent:

```
lenient = read_network(fixture_path("xdsl/equation_node.xdsl"); strict=false)
[v.id for v in lenient.variables], lenient.extras[:skipped]
```

```
([:Rain, :Wet], Dict{String, Any}[Dict("id" => "Runoff", "reason" => "unsupported
node type <equation>", "type" => "equation"), Dict("id" => "Flood", "reason" =>
"parent Runoff was skipped", "type" => "cpt")])
```

## 8 The package's own JSON

`write_ir_json` serialises the IR with a fixed key order; the test suite keeps one such golden file per fixture. Tables are stored with their dimensions and values in Julia column-major order.

```
write_ir_json(joinpath(dir, "asia.bnir.json"), asia["bif"])
println(join(readlines(joinpath(dir, "asia.bnir.json"))[1:22], "\n"))
```

```
{
  "format": "bnir",
  "version": 1,
  "name": "unknown",
  "source_format": "bif",
  "source": "../test/fixtures/bif/asia.bif",
  "variables": [
    {
```

```

    "id": "asia",
    "title": "asia",
    "kind": "chance",
    "states": ["yes", "no"],
    "parents": [],
    "deterministic": false,
    "table": {
      "dims": [2],
      "values": [0.01, 0.99]
    },
    "position": null,
    "comment": "",
    "extras": {}
  },

```

## 9 Summary

Six interchange formats read into one `NetworkIR`, and any IR writes back out to any of them; `differences` and `isequivalent` compare two IRs, `joint_distribution` and `marginal` give a brute-force oracle, and `strict=false` drops node types the IR cannot represent. The next vignette, *Axis conventions*, takes the one thing that must be right for any of this to mean anything, the order in which each format flattens a conditional probability table, and shows the same network in all six files entry by entry.

## 10 References

- Lauritzen, Steffen L., and David J. Spiegelhalter. 1988. "Local Computations with Probabilities on Graphical Structures and Their Application to Expert Systems." *Journal of the Royal Statistical Society, Series B* 50 (2): 157–224. <https://doi.org/10.1111/j.2517-6161.1988.tb01721.x>.
- Norsys Software Corp. 2024. *Netica File Formats*. [https://www.norsys.com/WebHelp/NETICA/X\\_File\\_Formats.htm](https://www.norsys.com/WebHelp/NETICA/X_File_Formats.htm).
- Scutari, Marco. 2010. "Learning Bayesian Networks with the bnlearn R Package." *Journal of Statistical Software* 35 (3): 1–22. <https://doi.org/10.18637/jss.v035.i03>.